

KYAMBOGO UNIVERSITY

FACULTY OF ENGINEERING · DEPARTMENT OF ELECTRICAL AND ELECTRONICS

TETE 3102: MICROCONTROLLERS AND APPLICATIONS

LABORATORY REPORT: TASK 8

Advanced Driver Architectures: Interrupt, DMA & Execution Profiling

Mukama Robert (26/U/ETE/21285/PE)
Nabudere Zerubabel (25/U/ETE/05465/PE)
Nanfuka Shakirah (25/U/ETE/05468/PE)
Kidega Wilkie Cephas (25/U/ETE/05460/PE)

Group Number: Group 4
Platforms: STM32F411RE & Arduino Uno
Course Supervisor: Dr. Mugerwa Dickson PhD

OBJECTIVE & LAB FOCUS

The objective of this laboratory task is to benchmark the execution latency, CPU utilization, and bus timing of the STM32 HAL I^2C driver across three operating paradigms: Polling (blocking), Interrupt (IT, non-blocking), and Direct Memory Access (DMA). By instrumenting firmware with GPIO toggling and capturing traces on an oscilloscope, students measure the exact CPU blocking duration versus the physical wire transfer time at both 100 kHz and 400 kHz.

1 Driver Paradigms & Firmware Implementation

1.1 1. Polling Mode Implementation (Baseline)

In polling mode, the CPU core is completely stalled inside a loop checking hardware status registers until all bytes have moved or a timeout expires.

Listing 1: Polling Mode with GPIO Benchmarking Pin

```
/* Benchmarking pin configured on PA5 (Nucleo green LED) */
HAL_GPIO_WritePin(GPIOA, GPIO_PIN_5, GPIO_PIN_SET); // HIGH = CPU Busy

HAL_I2C_Mem_Read(&hi2c1, DS3231_ADDR, 0x00, I2C_MEMADD_SIZE_8BIT, rtc_buf, 3, 100);

HAL_GPIO_WritePin(GPIOA, GPIO_PIN_5, GPIO_PIN_RESET); // LOW = CPU Free
```

1.2 2. Interrupt (IT) Mode Implementation

In interrupt mode, the HAL function configures peripheral control registers and enables the `I2C_IT_EVT` and `I2C_IT_ERR` interrupts, returning control to the main thread almost immediately.

Listing 2: Interrupt Mode Implementation with Non-Blocking Callbacks

```
/* Global / static buffer to prevent stack variable destruction */
static uint8_t rtc_buf[3];
volatile uint8_t rtc_rx_complete = 0;

/* Main execution dispatch */
HAL_GPIO_WritePin(GPIOA, GPIO_PIN_5, GPIO_PIN_SET);
HAL_I2C_Mem_Read_IT(&hi2c1, DS3231_ADDR, 0x00, I2C_MEMADD_SIZE_8BIT, rtc_buf, 3);
HAL_GPIO_WritePin(GPIOA, GPIO_PIN_5, GPIO_PIN_RESET); // Returns immediately!

/* Callback executed in ISR context when all 3 bytes are transferred */
```

```
void HAL_I2C_MemRxCpltCallback(I2C_HandleTypeDef *hi2c) {
    if (hi2c->Instance == I2C1) {
        rtc_rx_complete = 1;
    }
}
```

1.3 3. Direct Memory Access (DMA) Mode Implementation

DMA routes data directly between the I^{2}C Data Register (`I2C_DR`) and internal SRAM using dedicated DMA streams (`DMA1_Stream0` for `I2C1_RX` and `DMA1_Stream6` for `I2C1_TX`).

Listing 3: DMA Mode Implementation

```
HAL_GPIO_WritePin(GPIOA, GPIO_PIN_5, GPIO_PIN_SET);
HAL_I2C_Mem_Read_DMA(&hi2c1, DS3231_ADDR, 0x00, I2C_MEMADD_SIZE_8BIT, rtc_buf, 3);
HAL_GPIO_WritePin(GPIOA, GPIO_PIN_5, GPIO_PIN_RESET);
```

2 Experimental Oscilloscope Measurements & Profiling

An oscilloscope Channel 1 was connected to the GPIO benchmark pin (PA5) to measure CPU execution time, while Channel 2 monitored SCL to measure the physical bus activity duration. The transaction transfers 3 RTC bytes (Seconds, Minutes, Hours) following a 1-byte register pointer.

Table 1: CPU Execution Latency vs. Bus Transfer Duration Across Driver Modes

Driver Mode	Bus Freq.	CPU Return Time (t_{CPU})	Bus Wire Time (t_{bus})	CPU Load %	Operational Behavior
Polling	100 kHz	542.8 μs	541.2 μs	$\approx$ **100**%	CPU blocked for entirety of the transfer.
Interrupt (IT)	100 kHz	**14.2** μs	541.2 μs	$\approx$ **2.6**%	Returns in 14 μs; ISR handles byte transitions.
DMA	100 kHz	**9.8** μs	541.2 μs	$\approx$ **1.8**%	Returns in under 10 μs; DMA controller moves bytes.
Polling	400 kHz	138.4 μs	136.5 μs	$\approx$ **100**%	4$\times$ speedup on physical bus, CPU still 100% trapped.
Interrupt (IT)	400 kHz	**14.1** μs	136.5 μs	$\approx$ **10.3**%	Return is instant; ISR overhead is higher percentage.
DMA	400 kHz	**9.7** μs	136.5 μs	$\approx$ **7.1**%	Minimal CPU footprint; maximum wire throughput.

THEORETICAL FOUNDATION & PROTOCOL RULES

Analysis of the Speedup and Physical Limits: Switching from 100 kHz to 400 kHz reduces the physical bus transfer time by almost exactly a factor of 4 (from 541.2 μs down to 136.5 μs). However, the limits to increasing bus speed are constrained by physical physics:

1. **Bus Capacitance and Rise Time (t_r):** Higher clock frequencies require lower pull-up resistances ($R_p \leq 2.2$ kΩ) to meet the 300 ns rise time threshold.
2. **Slave Clock Stretching Limits:** Many sensors cannot process internal state transitions fast enough to sustain high data rates without stretching SCL, which stalls the master regardless of configured baud rates.
3. **DMA Overhead for Tiny Payloads:** For small 3-byte transfers, DMA setup latency ($\approx 10\,\mu$s) is comparable to IT dispatch latency. DMA's massive advantage occurs during long payload streaming (e.g. 1024-byte graphical OLED frames).

3 Extension: Inter-Peripheral Loopback (I2C1 Master to I2C2 Slave)

The STM32F411RE features three independent I^{2}C hardware peripherals. In this extension, I2C1 was configured as master (PB8/PB9) and I2C2 was configured as slave on pins PB10 (I2C2_SCL) and PB3 (I2C2_SDA).

- **Pin Connections:** PB8 was jumpered to PB10 (SCL), and PB9 was jumpered to PB3 (SDA), with common 2.2 kΩ pull-ups to 3.3 V.
- **CubeMX Configuration:** I2C2 was enabled in slave mode with primary slave address set to `0x32` (`0x64` shifted).
- **Communication Loop:** The master calls `HAL_I2C_Master_Transmit(&hi2c1, 0x64, tx_data, 4, 100)`, while the slave executes an interrupt receive via `HAL_I2C_Slave_Receive_IT(&hi2c2, rx_data, 4)`.
- **Outcome:** 100% reliable communication was verified within a single microcontroller without external peripheral hardware.

4 Conclusion

Task 8 demonstrates that while polling is straightforward to implement, Interrupt and DMA modes liberate over 97% of the CPU core's computational capacity during I^{2}C transactions, making them the imperative choice for real-time control, sensor telemetry, and graphics rendering.